

Communication in Distributed Architecture

Optimizing Point-to-Point Data Transfer Throughput

Eddie Odiwuor Ogendero

February 1, 2026

Abstract

This research investigates the upper limits of data transfer rates between two point nodes. The primary objective is to evaluate the impact of different communication protocols, message sizes, and Modular Component Architecture (MCA) tuning on total bandwidth and latency.

Contents

1	Introduction	3
2	High Performance Parallelism	3
2.1	Message Passing Interface (MPI)	3
2.1.1	The OpenMPI Architecture	4
2.1.2	Communication Protocols	4
2.2	Open Multi-Processing (OpenMP)	5
2.3	Difference between MPI and OpenMP	5
3	Networking Foundation	5
3.1	Resumable/Partial downloads	5
3.1.1	Range	5
3.1.2	Accept-Range	6
3.1.3	Content-Range	6
3.1.4	If-range	6
3.2	Resumable/Multipart Uploads	6
3.2.1	Stages	6
3.2.2	Advantages	7
3.2.3	Best practises	7
3.2.4	Standard protocols	7
4	Kernel-level Optimization	7
4.1	IO uring	7
4.2	Kernel TLS (KTLS)	7
5	Synthesis & Benchmarking	7
5.1	Broadcaster	7
6	Experimental Methodology	7
6.1	Hardware Setup	7
6.2	Software Environment	7
6.3	Software Environment	8
6.4	Benchmark Design	8

7	Optimization Strategies	8
7.1	Non-Blocking Communication	8
7.2	Remote Memory Access (RMA)	8
7.3	MCA Parameter Tuning	9
8	Results and Analysis	9
8.1	Bandwidth Analysis	9
8.2	Mathematical Performance Model	9
8.3	Comparison of Protocols	9
9	Conclusion	9

1 Introduction

The purpose of this research is to learn about the complexity of data movement problem in distributed systems.

Optimizing data transfer between two nodes involves minimizing the overhead introduced by the operating system, the network stack, and the physical limitations of the hardware.

The research will be structured in terms of these core concepts

- **High Performance Parallelism:** Message Passing Interface, point-to-point vs Collective communication.
- **Networking Foundations:** TCP/IP stack, Network fallacies, Chunking and Retries.
- **Kernel-level optimization:** io_uring, ktls (Kernel TLS) and zero-copy techniques.
- **Synthesis & Benchmarking:** Building the Broadcaster and measuring throughput vs latency.

2 High Performance Parallelism

2.1 Message Passing Interface (MPI)

Industry standard specification used for communication between processes in parallel computing

MPI attempts to solve the following problems:

- **Portability:**
Each vendor had propriety library for message passing thus requiring rewrites for different use cases.
Solution - *MPI* provides a standard API provided an MPI is installed (OpenMPI, MPICH)
- **Memory wall (*Distributed Memory Management*):**
Most large scale computers are distributed memory systems. Moving data between isolated memory manually is complex and error-prone.
Solution - *MPI* treats communication as explicit messages. *MPI* uses objects called *communicators* and *groups* to define which collection of processes may communicate with each other.

Provides functions that handle networking, buffering and memory copying.

- **Sender:** A process calls *MPI_Send()* (or a variant) to transmit a message.
- **Receiver:** A process calls *MPI_Recv()* to accept the message.
- **Message Envelope:** Data buffer (address, type, count), Source/destination ranks, Message tag, Communicator (*MPI_COMM_WORLD* by default)
- **Complexity of collective operations:**
Parallel algorithms often send pieces of data or collects results from multiple end points. Writing *one-to-many* or *many-to-one* patterns is tedious and usually inefficient.

Solution - *MPI* provides collective communication through built in functions.

- **Broadcaster** - sends data from one process to all.

- **Scatter/Gather** - Splitting a large vector into chunks for different processes or reassembling them.
- **Reduction** - performing calculations across all processors and return a single result.
- **Hardware heterogeneity:**
Computing clusters consists of different machines with different architecture e.g *big/little endian*.
Solution - MPI uses opaque datatypes and handles translation between different machines e.g. *(4, MPI_INT) a vector with 4 integer values*

2.1.1 The OpenMPI Architecture

OpenMPI is based on the Modular Component Architecture is a system that breaks down a software into independent, interchangeable modules, each encapsulating specific functionality. It is a series of projects, frameworks, components, and modules that are assembled at run-time to create an *MPI* implementation.

Each framework typically has one or more components that are used at run-time. **Point-to-point Management** Layer sits between the *MPI* API and the lower-level network transport layers. It manages message matching, ordering, and delivery guarantees required by the *MPI* standard. It decides how and through which transport a message will be sent. The *ucx pml* sends messages across **InfiniBand-based networks**.

The *btl* framework is used by the *MPI* layer to send bytes across different types underlying networks. The *tcp btl*, for example, sends messages across TCP-based networks

2.1.2 Communication Protocols

MPI provides three different communication methods that *MPI* processes can use to communicate with each other.

- **Point-to-Point Communication** - the most used communication method. It involves the transfer of a message from one process to a particular process in the same communicator. *MPI* provides blocking (*synchronous*) and non-blocking (*asynchronous*) point-to-point communication.
For the *blocking* communication, an *MPI* process *sends a message to another MPI process and waits until the receiving process completely and correctly receives the message* before it continues its work.
On the other hand, a sending process using *non-blocking* communication *sends a message to another MPI process and continues its work without waiting* to ensure that the message has been correctly received by the receiving process.
- **Collective Communication** - a process broadcasts a message to all processes in the same communicator including itself.
- **One-sided communication** - a process can directly access the memory space of another process without involving it.

```

1      /* Standard send in MPI. Behind the scenes, it will
2         issue a buffered send MPI_Bsend or a synchronous
3         send MPI_Ssend, based on whether the buffer attached
           contains enough free space for the message. */
int MPI_Send(const void* buffer, int count,
             MPI_Datatype datatype, int recipient, int tag,
             MPI_Comm communicator);

```

```

4      /* Asynchronous blocking send ('B' == buffered); it
       will block until a copy of the buffer passed is made
       . MPI will send the buffer copied at a later point
       */
5      int MPI_Bsend(const void* buffer, int count,
        MPI_Datatype datatype, int recipient, int tag,
        MPI_Comm communicator);
6
7      /* Synchronous blocking send; it will block until the
       recipient has received the message. Ensure the
       buffer and be used safely once MPI_Ssend return */
8      int MPI_Ssend(const void* buffer, int count,
        MPI_Datatype datatype, int recipient, int tag,
        MPI_Comm communicator);
9
10     /* Standard blocking send ('R' == ready); it will block
       until the recipient has received the message.
       Requires the recipient to have issued the
       corresponding receive routine (MPI_Recv or MPI_Irecv
       ) before MPI_Rsend is invoked. */
11     int MPI_Rsend(const void* buffer, int count,
        MPI_Datatype datatype, int recipient, int tag,
        MPI_Comm communicator);

```

2.2 Open Multi-Processing (OpenMP)

An application programming interface (API) that supports multi-platform shared-memory multiprocessing programming in C, C++, and Fortran. It allows developers to write parallel code easily and efficiently by adding simple compiler directives to their existing code.

Utilizes the **Pragma** directive to allow the compiler to run the code sequentially if it doesn't support *OpenMP*.

```

1      #pragma omp [construct] [clause]

```

2.3 Difference between MPI and OpenMP

MPI	OpenMP
Supports parallel computation for distributed-memory and shared-memory systems.	Supports parallel computation for shared-memory systems only.
A process-based parallelism.	A thread-based parallelism.
Each process has its own memory space and executes independently from the other processes.	Threads share the same resources and access shared memory.
Processes exchange data by passing messages to each other.	There is no notion of message-passing. Threads access shared memory.
Process creation overhead occurs one time.	It depends on the implementation. More overhead can occur when creating threads to join a task.

Table 1: Comparison between MPI and OpenMP

3 Networking Foundation

3.1 Resumable/Partial downloads

To support **Partial / Resumable downloads** they need to be supported by the server(sender).

3.1.1 Range

Indicates what part of a resource that the server should return

Status code:

- **206** : *Partial content*, server can send the request.
- **416** : *Range not satisfiable*, ranges are invalid.
- **200** : Server doesn't support range request or request ignored

The header requests one or more subranges of selected data, *similar to skips in audio or video*.

```
1      Range: <unit>=<start-end>
2      Range: <unit>=<n->           // start from chunk n
      till the end
3      Range: <unit>=<-n>           // select the last n
      chunks
4      Range: <unit>=<start-end>, <start-end>, <start-end> //
      select multiple chunks from different position
5      Range: <unit>=<start-end>, <-end>
```

3.1.2 Accept-Range

Indicates if a server accepts range requests

```
1      Accept-Range: none
2      Accept-Range: <unit> // e.g. bytes
```

3.1.3 Content-Range

Indicates where the content of a response belongs in relation to a complete resource

```
1      Content-Range: <unit> <range>/<size> // status : 206
2      Content-Range: <unit> <range>\* // Length unknown
3      Content-Range: <unit> *\<size> // 416 : not in range
```

Server service requests in order but may reorder ranges in sequential order in order to not buffer an entire response on disk

3.1.4 If-range

Makes range requests conditional, **206**=*fulfilled* and **200**=*ignored*.

Header can use either *last-modified* or *Etag* but not both. It can be used to check for change since last download.

3.2 Resumable/Multipart Uploads

The process involve breaking a large file into smaller chunks and sending the individually.

Resumability is achieved by tracking which chunk have been successfully acknowledged by the server.

If a receiver fails, sender only sends the missing chunks instead of the entire file

3.2.1 Stages

Initiation:

Sender sends a post request to the receiver(s) to start an upload.

Receiver returns a unique *upload-id* and prepares storage for incoming parts

Part upload:

File is sliced into chunks each uploaded with a *part-number*, *upload-id*

Finalization:

Once all parts are uploaded, sender sends a completion request listing all *Etags*.

Receiver stitches all pieces together and send a confirmation.

3.2.2 Advantages

Parallel streams - sender open 4-8 concurrent http connections to upload different parts fully utilizing available bandwidth.

Memory efficiency - remove need of loading large amount of data in ram, sliced small chunks keeps memory footprint low.

Error isolation - failure doesn't affect the whole upload process just the slice.

3.2.3 Best practises

Exponential backoff:

If a part fails, don't retry immediately have increasing wait times before next trial (*1s* - *2s* - *3s*) as a form of network congestion control.

MD5 validation:

Always send a MD5 header with each part, it lets the receiver verify the integrity before acknowledging.

Cleanup process:

Add lifecycle rules to deal with unfinished uploads(e.g. delete) after a specified time period.

Dynamic chunking:

Adjust chunk size based on network resiliency; *5MB(mobile)*, *25MB(Fibre)*.

3.2.4 Standard protocols

- AWS S3 Multipart.
- tus.io

4 Kernel-level Optimization

4.1 IO uring

4.2 Kernel TLS (KTLS)

5 Synthesis & Benchmarking

5.1 Broadcaster

6 Experimental Methodology

6.1 Hardware Setup

Describe the CPU, RAM, and NIC (Network Interface Card) of the two nodes.

6.2 Software Environment

The software stack was selected to ensure high-performance execution and minimal overhead during point-to-point data transfers. The environment is detailed as follows:

6.3 Software Environment

The software stack was selected to ensure high-performance execution and minimal overhead during point-to-point data transfers. The environment is detailed as follows:

- **Operating System:** The experiments were conducted on a Windows Subsystem for Linux running Ubuntu 24.04.3 LTS with the **Kernel version 6.6.87.2-microsoft-standard-WSL2**. To ensure maximum throughput, we verified that `sysctl` parameters for TCP/IP (such as `net.core.rmem_max` and `net.core.wmem_max`) were tuned to support large window sizes for high-bandwidth networks.
- **MPI Implementation:** We utilized **Open MPI v5.0.9**. The library was built against external **PMIx v5.0.9** and **PRRTE v3.0.12** dependencies to ensure a modern, modular process management layer. For intra-node communication within the WSL2 environment, the Modular Component Architecture (MCA) was verified to prioritize the `sm` (shared memory) and `self` Byte Transport Layers (BTL). This configuration enables low-latency, high-bandwidth data movement via shared memory regions, bypassing the network stack for local processes. Additionally, the `tcp` BTL was configured for management and coordination across the virtualized network interface.
- **Runtime Infrastructure:** To support the Open MPI 5.0 modular architecture, we deployed the **PMIx (Process Management Interface Exascale) v5.0.9** and **PRRTE (PMIx Reference RunTime Environment) v3.0.12** frameworks. These components were manually compiled and linked to provide a robust distributed process management layer, facilitating efficient process wire-up and synchronization during the `MPI_Init` phase.
- **Compiler and Flags:** All C source files were compiled using the **GCC 14.2.0** suite. To achieve "wire-speed" performance and ensure the CPU does not become the bottleneck during data packing/unpacking, the following optimization flags were applied:
 - `-O3`: Enables aggressive code optimizations including loop vectorization.
 - `-march=native`: Instructs the compiler to generate instructions specific to the host CPU architecture.
 - `-mtune=native`: Optimizes the generated code for the local processor's pipeline.

- `-ffast-math`: Relaxes some strict IEEE floating-point compliance to further speed up arithmetic during data processing.

6.4 Benchmark Design

Description of the Ping-Pong benchmark logic using `MPI_Send` and `MPI_Recv`.

7 Optimization Strategies

7.1 Non-Blocking Communication

Implementation of `MPI_Isend` and `MPI_Irecv` to hide latency.

7.2 Remote Memory Access (RMA)

One-sided communication using `MPI_Put` and `MPI_Get`.

7.3 MCA Parameter Tuning

Analysis of specific runtime flags like `btl_openib_eager_limit` and `pml_ucx`.

8 Results and Analysis

8.1 Bandwidth Analysis

Present the bandwidth curve relative to message size.

8.2 Mathematical Performance Model

The observed bandwidth $B(L)$ for a message of size L is modeled as:

$$B(L) = \frac{L}{\alpha + \beta L}$$

Where α represents latency and β represents the reciprocal of the peak bandwidth.

8.3 Comparison of Protocols

Graph comparing Blocking vs. Non-blocking vs. RMA.

9 Conclusion

Summary of the most effective configuration and final throughput achieved.

References

- [1] The Open MPI Project. *Open MPI: A High Performance Message Passing Library*. <https://www.open-mpi.org/>
- [2] MPI Basics by Tom Nurkkalla. Introduction to distributed computing with MPI. <https://www.youtube.com/watch?v=c0C9mQaxsD4>

- [3] Introduction to Parallel Programming using MPI. [*https://arcca.github.io/intro-mpi/*](https://arcca.github.io/intro-mpi/)
- [4] RookieHPC, learn about High-Performance Computing. [*https://rookiehpc.org/*](https://rookiehpc.org/)